

Robot Trajectory Optimization

2D Robot Path Planning via Nonlinear Model Predictive Control
Active-Set QP + SQP Trajectory Optimization with Real-Time Visualization

Emin Hadžiabdić

Faculty of Electrical Engineering (ETF), University of Sarajevo

Course: Numerical Optimization — Prof. Dr. Izudin Džafić

Academic Year 2025/26

Abstract

This document describes **RobotTrajectoryOpt**, a desktop C++ application that plans and simulates the motion of a 2D mobile robot (kinematic bicycle model) using a receding-horizon **Model Predictive Controller (MPC)**. At every control step the controller re-linearizes the vehicle dynamics around the current nominal trajectory, constructs a **Quadratic Program (QP)** with explicit inequality constraints (steering bounds, acceleration bounds, velocity bounds, and obstacle avoidance), and solves it with a full **Active-Set QP solver** that uses sparse indefinite **LDL^T** factorization at every inner iteration. Iterating this linearize-and-solve procedure constitutes a **Sequential Quadratic Programming (SQP)** scheme. The controller drives the vehicle along smooth quintic reference paths while respecting actuator limits and avoiding obstacles through linearized inequality constraints with slack variables. The application ships with an interactive GUI, built on the **natID** GUI framework, that renders the path, the vehicle, the predicted horizon, live solver diagnostics, and time-series control plots. Version 2.0 replaces the earlier KKT saddle-point solver with a proper active-set method that handles inequality constraints as explicit QP rows, and adds velocity bounds, stagnation detection, QP failure recovery, and extended solver settings.

Contents

1	Introduction	3
1.1	What This Project Is	3
1.2	Why Optimization Instead of A*	3
1.3	Project Goals	3
1.4	Relationship to the Original Proposal	3
1.5	What Is New in Version 2.0	4
2	Logic: Mathematical and Physical Model	4
2.1	Vehicle Model: Kinematic Bicycle	4
2.2	Decision Vector	5
2.3	Cost Function	5
2.4	Reference Path: Quintic Polynomials	5
2.5	Vehicle Dynamics as Equality Constraints	6
2.6	Actuator and Velocity Limits as QP Inequality Constraints	6
2.7	Obstacle Avoidance Constraints	7
2.8	The Full QP Subproblem	7
2.9	Active-Set QP Solver	7
2.10	SQP Algorithm	8
2.11	Receding Horizon, Warm-Starting, and Robustness Mechanisms	8

3	Usage	9
3.1	Purpose of the Application	9
3.2	What It Shows	9
3.3	Building and Running	9
3.4	Typical Session	9
4	Features	10
4.1	Scenarios	10
4.2	Adding a New Scenario	10
4.3	Solver Controls	10
4.4	Multi-Language Support	11
4.5	Diagnostics and Logging	11
5	GUI Application Introduction	11
6	GUI Features	11
6.1	Toolbar	11
6.2	Path Tracking Canvas	11
6.3	Time-Series Canvases	12
6.4	Sidebar: MPC Controls	12
6.5	Sidebar: Live Metrics	13
6.6	Settings Dialog	13
7	Verification: Does the Implementation Match the Proposal?	13
7.1	Control Flow Audit	14
7.2	What “No Cheating” Means	15
7.3	Mathematical Verification	15
7.4	Conclusion	16
8	Conclusions	16
8.1	What the Project Demonstrates	16
8.2	Problems It Solves	16
8.3	Lessons Learned	16
8.4	Possible Future Work	17

1 Introduction

1.1 What This Project Is

RobotTrajectoryOpt is a from-scratch implementation of trajectory optimization for a 2D ground robot, built as the term project for the *Numerical Optimization* course. Rather than solving the path-planning problem with a discrete search method such as A*, the project treats motion planning as a continuous constrained optimization problem: at every instant, find the sequence of steering and acceleration commands over a short future horizon that keeps the robot as close as possible to a target path, while never violating the vehicle’s physical limits.

This is the same class of controller used in production automotive and robotics systems (Model Predictive Control), scaled down to a self-contained educational implementation where every matrix, every constraint row, and every solver call is visible and traceable.

1.2 Why Optimization Instead of A*

A* and similar grid-search planners answer the question “what sequence of cells connects A to B” on a discretized map. They do not natively reason about vehicle dynamics (a car cannot rotate in place, cannot exceed a maximum steering angle, and cannot change speed instantaneously), so their output usually needs a separate tracking controller layered on top. RobotTrajectoryOpt instead poses path following directly as a **constrained quadratic program**: the dynamics themselves are constraints of the optimization, so the controller can never propose a command the robot could not physically execute. This unifies planning and control into a single mathematical formulation.

1.3 Project Goals

- Formulate the vehicle’s motion as a discrete-time **kinematic bicycle model**.
- Build a **quadratic cost function** that penalizes deviation from a reference path and penalizes aggressive control effort.
- Linearize the nonlinear dynamics at every SQP iteration and express them as **equality constraints** in a sparse constraint matrix.
- Encode actuator limits (steering, acceleration, velocity) and obstacle avoidance as **explicit inequality constraint rows** inside the QP.
- Solve the resulting QP at each SQP iteration using a **full active-set method** with warm-starting, Tikhonov regularization, and sparse LDL^T factorization.
- Wrap the solver in an interactive real-time **GUI** for experimentation, tuning, and visual verification.

1.4 Relationship to the Original Proposal

The original project proposal (*2D Robot Path Planning using Trajectory Optimization (MPC)*, May 2026) outlined a plan to adapt two open-source C++ MPC references and re-implement their mathematical core using custom Dense/Sparse matrix structures from the course’s matrix library, and to visualize the result with the **natID** GUI framework. The delivered implementation follows this plan faithfully: the two referenced repositories are retained, unused, under **res/mpc/** purely as background reference material (they are not compiled into the application — see Section 7), while the actual solver in **src/** is a clean re-implementation built entirely on **natID**’s **dense::DblMatrix** and **sparse::IDblMatrix/sparse::ISolver** types, exactly as proposed.

1.5 What Is New in Version 2.0

Version 2.0 introduces several significant improvements over v1.0:

- **Active-Set QP solver** (`MpcActiveSetQp.h`): Replaces the earlier KKT saddle-point assembly/solve path (`MpcKkt.h`, `MpcKktSolver.h` — now deleted). The new solver handles inequality constraints as explicit QP rows with warm-starting, Tikhonov regularization, duplicate row detection, dual tolerance, and tag-based warm-start matching.
- **Explicit inequality constraints** (`MpcInequality.h`): Steering bounds, acceleration bounds, and velocity bounds are encoded as proper linear inequality rows in the QP, replacing the previous post-hoc clamping approach.
- **Velocity bounds**: $v \geq 0$ and $v \leq v_{\max}$ ($v_{\max} = 3.0$ m/s) are enforced as QP constraints.
- **Obstacle avoidance as inequality rows**: Linearized obstacle distance constraints and slack non-negativity are formulated as explicit QP inequality rows.
- **Warm-start t=0 snap**: The warm-start trajectory's initial state is snapped to the actual vehicle telemetry before passing to the SQP solver.
- **Fresh reference every step**: The reference trajectory is regenerated from the vehicle's current position at each step, preventing warm-start drift.
- **Stagnation detection**: When the SQP converges in very few iterations with near-zero update, the nominal trajectory is reset on the next step.
- **QP failure recovery**: When the SQP solver fails, the engine falls back to polynomial reference trajectory with feedforward controls.
- **Extended settings dialog**: Added α (step size) and `maxActiveSetIter` to the settings dialog, with translation support in all 6 UI languages.

2 Logic: Mathematical and Physical Model

2.1 Vehicle Model: Kinematic Bicycle

The robot is modeled as a kinematic bicycle with state $\mathbf{s} = (x, y, \psi, v)$ — planar position, heading, and longitudinal speed — and controls $\mathbf{u} = (\delta, a)$ — steering angle and acceleration. The continuous-time equations are discretized with a fixed step Δt :

$$x_{t+1} = x_t + v_t \cos(\psi_t) \Delta t \quad (1)$$

$$y_{t+1} = y_t + v_t \sin(\psi_t) \Delta t \quad (2)$$

$$\psi_{t+1} = \psi_t + \frac{v_t}{L_f} \delta_t \Delta t \quad (3)$$

$$v_{t+1} = v_t + a_t \Delta t \quad (4)$$

where L_f is the distance from the vehicle's center of mass to the front axle (an effective wheelbase term). In the implementation:

$$\Delta t = 0.1 \text{ s}, \quad L_f = 0.5 \text{ m}, \quad N = 20 \text{ (horizon steps, i.e. a 2.0 s look-ahead).}$$

2.2 Decision Vector

At every solve, the controller optimizes over a stacked vector containing the entire predicted state and control sequence, plus slack variables for obstacle avoidance:

$$z = \left[\underbrace{x_0, \dots, x_{N-1}}_N, \underbrace{y_0, \dots, y_{N-1}}_N, \underbrace{\psi_0, \dots, \psi_{N-1}}_N, \underbrace{v_0, \dots, v_{N-1}}_N, \underbrace{\delta_0, \dots, \delta_{N-2}}_{N-1}, \underbrace{a_0, \dots, a_{N-2}}_{N-1}, \underbrace{\sigma_0, \dots}_{N \cdot m} \right]$$

where m is the number of obstacle-slack slots reserved per step (2 in the shipped build). For $N = 20$ this gives $4N + 2(N - 1) + 2N = 80 + 38 + 40 = 158$ decision variables (realized by `MpcLayout`).

2.3 Cost Function

The controller minimizes a purely quadratic tracking-and-effort cost, diagonal in z :

$$J(z) = \sum_{t=0}^{N-1} \left[q_x(x_t - x_t^{\text{ref}})^2 + q_y(y_t - y_t^{\text{ref}})^2 + q_\psi(\psi_t - \psi_t^{\text{ref}})^2 + q_v(v_t - v_t^{\text{ref}})^2 \right] + \sum_{t=0}^{N-2} \left[r_\delta \delta_t^2 + r_a a_t^2 \right] + \sum w_{\text{slack}} \sigma^2 \quad (5)$$

Because the layout is diagonal, this is implemented as a diagonal Hessian $H = \text{diag}(h_i)$ and a linear term $g = -Hz^{\text{ref}}$, so that

$$J(z) = \frac{1}{2} z^\top H z + g^\top z + \text{const} \equiv \frac{1}{2} (z - z^{\text{ref}})^\top H (z - z^{\text{ref}}) \quad (6)$$

The tuned weights used in the shipped controller are:

Symbol	Value	Role
q_x	0.5	Weak — x is largely a consequence of speed, not directly steered
q_y	50.0	Strong — dominant term for lateral path tracking
q_ψ	25.0	Strong — keeps heading aligned with the path tangent
q_v	1.0	Weak — lets the solver trade velocity freely to aid tracking
r_δ	0.5	Penalizes aggressive steering
r_a	0.5	Penalizes aggressive acceleration
w_{slack}	8000.0	Very strong — makes obstacle-avoidance slack expensive

The large gap between q_y, q_ψ and q_v is a direct, documented consequence of early tuning: with a high velocity weight the solver preferred to hold speed constant over tracking the path, so the weights were rebalanced toward lateral/heading accuracy.

2.4 Reference Path: Quintic Polynomials

The desired road/path is a fifth-degree (quintic) polynomial $y = f(x)$:

$$y(x) = c_0 + c_1 x + c_2 x^2 + c_3 x^3 + c_4 x^4 + c_5 x^5, \quad \frac{dy}{dx}(x) = c_1 + 2c_2 x + 3c_3 x^2 + 4c_4 x^3 + 5c_5 x^4 \quad (7)$$

A quintic (rather than a lower-order cubic) is used specifically so that both the path's *value* and its *slope* can be pinned at the start and end of a maneuver. Choosing coefficients so that $y(0) = 0$ and $y'(0) = 0$ makes the reference heading match the vehicle's initial heading ($\psi_0 = 0$) exactly,

which removes an instantaneous heading mismatch that a non-zero-slope cubic would otherwise create.

The reference is projected forward from the vehicle’s current position along its own current heading direction, so that the N -step reference grid always represents *where the vehicle will actually be*, not where a nominal constant-speed vehicle would be. In final product, the reference trajectory is regenerated from the vehicle’s current position at every step (`_zNomInitialized` is forced to `false` before each solve), and the warm-start trajectory’s $t=0$ state is snapped to the actual vehicle telemetry. Past the designed end of the maneuver (`trackLength`), the reference degrades gracefully into a straight-line continuation tangent to the polynomial, so the tracker never chases an unboundedly steepening curve.

A feedforward steering estimate is also computed from the reference curvature:

$$\delta_{\text{ff}} = \arctan\left(\frac{L_f \cdot \Delta\psi}{v \cdot \Delta t}\right) \quad (8)$$

where $\Delta\psi = \psi_{\text{ref}}(t+1) - \psi_{\text{ref}}(t)$, providing a good initial guess for the steering variable.

2.5 Vehicle Dynamics as Equality Constraints

The nonlinear dynamics are not solved exactly; they are re-linearized every SQP iteration around a current nominal trajectory $\bar{z}$ and expressed as a linear equality constraint block $Az = b$. For each step $t = 0, \dots, N-2$, linearizing about $(\bar{\psi}_t, \bar{v}_t, \bar{\delta}_t)$ gives four constraint rows:

$$\begin{aligned} x_{t+1} - x_t - \Delta t \cos(\bar{\psi}_t) v_t + \Delta t \bar{v}_t \sin(\bar{\psi}_t) \psi_t &= \Delta t \bar{v}_t \sin(\bar{\psi}_t) \bar{\psi}_t \\ y_{t+1} - y_t - \Delta t \sin(\bar{\psi}_t) v_t - \Delta t \bar{v}_t \cos(\bar{\psi}_t) \psi_t &= -\Delta t \bar{v}_t \cos(\bar{\psi}_t) \bar{\psi}_t \\ \psi_{t+1} - \psi_t - \frac{\Delta t}{L_f} \bar{v}_t \delta_t - \frac{\Delta t}{L_f} \bar{\delta}_t v_t &= -\frac{\Delta t}{L_f} \bar{v}_t \bar{\delta}_t \\ v_{t+1} - v_t - \Delta t a_t &= 0 \end{aligned} \quad (9)$$

plus four rows locking the very first state (x_0, y_0, ψ_0, v_0) to the robot’s true, measured telemetry. These are the first-order (Jacobian) linearizations of the bicycle model around the current guess; because the guess is refreshed and the constraints re-linearized every iteration, iterating this process converges to a solution that (approximately) satisfies the true nonlinear dynamics — the defining idea of SQP.

2.6 Actuator and Velocity Limits as QP Inequality Constraints

Unlike v1.0, where actuator limits were enforced by post-hoc clamping, version 2.0 encodes all bounds as **explicit inequality constraint rows** inside the QP. These are built once per SQP call by `MpcInequality::buildBoundRows()` and remain constant across SQP iterations:

$$-\delta_{\text{max}} \leq \delta_t \leq \delta_{\text{max}}, \quad -a_{\text{max}} \leq a_t \leq a_{\text{max}}, \quad 0 \leq v_t \leq v_{\text{max}} \quad (10)$$

Each bound is represented as a single-row inequality $\mathbf{c}^\top z \geq b$:

Bound	Tag	Inequality Form
$\delta_t \geq -\delta_{\text{max}}$	$(0 \ll 24) \mid (t \ll 8)$	$1 \cdot \delta_t \geq -0.6$
$\delta_t \leq +\delta_{\text{max}}$	$(1 \ll 24) \mid (t \ll 8)$	$-1 \cdot \delta_t \geq -0.6$
$a_t \geq -a_{\text{max}}$	$(2 \ll 24) \mid (t \ll 8)$	$1 \cdot a_t \geq -3.0$
$a_t \leq +a_{\text{max}}$	$(3 \ll 24) \mid (t \ll 8)$	$-1 \cdot a_t \geq -3.0$
$v_t \geq 0$	$(4 \ll 24) \mid (t \ll 8)$	$1 \cdot v_t \geq 0$
$v_t \leq v_{\text{max}}$	$(5 \ll 24) \mid (t \ll 8)$	$-1 \cdot v_t \geq -v_{\text{max}}$

The tag encoding $(\text{kind} \ll 24) | (t \ll 8) | \text{idx}$ uniquely identifies each constraint row for the active-set warm-start matching across SQP iterations. The solver defaults are $\delta_{\max} = 0.6$ rad ($\approx 34.4^\circ$), $a_{\max} = 3.0$ m/s², and $v_{\max} = 3.0$ m/s.

A **post-hoc clamp safety-net** is also retained: after each active-set QP solve, the iterate is clamped to the bounds as a fallback. In practice this safety-net rarely fires because the explicit QP constraints already enforce the bounds.

2.7 Obstacle Avoidance Constraints

Obstacles are circular keep-out zones $(x_{\text{obs}}, y_{\text{obs}}, r)$ with an extra 0.3 m safety margin, i.e. an effective avoidance radius $R = r + 0.3$. For a point (x, y) close to an obstacle, the (nonlinear) avoidance condition

$$(x - x_{\text{obs}})^2 + (y - y_{\text{obs}})^2 \geq R^2$$

is linearized about the nominal predicted position $(\bar{x}, \bar{y})$ using its gradient $\nabla = 2(\bar{x} - x_{\text{obs}}, \bar{y} - y_{\text{obs}})$, and expressed as a linear inequality row with a per-step slack variable $\sigma \geq 0$:

$$\nabla_x x + \nabla_y y + \sigma \geq \nabla_x \bar{x} + \nabla_y \bar{y} + (R^2 - \|\bar{\mathbf{x}} - \mathbf{x}_{\text{obs}}\|^2) \quad (11)$$

In addition, each slack variable has a non-negativity constraint $\sigma \geq 0$. Both rows are built by `MpcConstraints::buildObstacleRows()` and injected into the QP as explicit inequality rows.

The heavy slack weight $w_{\text{slack}} = 8000$ in the cost makes the optimizer strongly prefer trajectories that keep σ at zero, which in turn discourages the nominal trajectory from lingering near an obstacle across SQP iterations. Far from any obstacle (nominal distance beyond $2R$) the slack collapses to zero and the constraint becomes inactive.

2.8 The Full QP Subproblem

At every SQP iteration the controller solves:

$$\min_z \frac{1}{2}(z - z^{\text{ref}})^\top H(z - z^{\text{ref}}) \quad \text{s.t.} \quad A(\bar{z})z = b(\bar{z}), \quad C_{\text{ineq}}z \geq d_{\text{ineq}} \quad (12)$$

where H is diagonal, A, b are the linearized dynamics/initial-state rows, and $C_{\text{ineq}}, d_{\text{ineq}}$ collect all inequality rows (bounds + obstacle rows). This is solved by the active-set QP solver.

2.9 Active-Set QP Solver

The active-set QP solver (`MpcActiveSetQp.h`) handles the inequality constraints by iteratively treating a subset of them as equalities (the *working set*), solving the resulting equality-constrained QP via the KKT system, and then adding/removing constraints based on feasibility and optimality:

1. **Initialize** the working set from the previous solve's active set (warm-start, matched by tag).
2. **Assemble** a sparse KKT system treating active inequalities as equalities:

$$\begin{bmatrix} H & A_{\text{eq}}^\top \\ A_{\text{eq}} & 0 \end{bmatrix} \begin{bmatrix} z \\ \lambda \end{bmatrix} = \begin{bmatrix} -g \\ b_{\text{eq}} \end{bmatrix}$$

where A_{eq} stacks the dynamics equality rows and the currently active inequality rows.

3. **Factorize and solve** with sparse indefinite LDL^T using `natID's` `sparse::createDblSolver` with `Symmetry::SymmetricIndef`, `SolverType::LDLT`, `Pivoting::DiagonalSinglePass`. Tikhonov regularization (adding $\epsilon = 10^{-8}$ to zero diagonal entries of H) prevents singularity.

4. **Check feasibility:** for each *inactive* inequality, test whether the proposed step would violate it. If so, find the blocking constraint and add it to the working set (partial step to the constraint boundary).
5. **Check optimality:** examine the Lagrange multipliers λ of the active constraints. If any $\lambda < -\epsilon_{\text{dual}}$ ($\epsilon_{\text{dual}} = 10^{-8}$), the corresponding constraint is redundant and is removed from the working set.
6. **Post-hoc clamp** as a safety-net.
7. **Converge** when no blocking constraint is found and all multipliers are non-negative, or when the maximum inner iteration count (`maxActiveSetIter` = 20) is reached.

Duplicate row detection prevents adding two rows with nearly identical coefficient vectors (within 10^{-12}) to the working set.

2.10 SQP Algorithm

The outer SQP loop re-linearizes the nonlinear dynamics at each iteration and calls the active-set QP:

1. Update the reference trajectory (quintic polynomial) from the vehicle’s current position.
2. Linearize the dynamics at z_{nom} to produce A, b .
3. Build obstacle inequality rows for this linearization.
4. Combine with bound rows (constant across iterations).
5. Solve the active-set QP.
6. Damp the step: $z \leftarrow z + \alpha \cdot (z_{\text{new}} - z)$ with $\alpha = 0.12$.
7. Check convergence: $\max_i |z_i^{\text{new}} - z_i| < \text{tol}$.
8. Track the best solution for rollback on failure.

Convergence is declared once $\max_i |z_i^{\text{new}} - z_i| < \text{tol}$ (default 2×10^{-3}); if the iteration limit (default 60) is reached first, the solver rolls back to the iterate with the smallest observed $\max |\Delta z|$ and returns that trajectory as a stable, if sub-converged, result.

2.11 Receding Horizon, Warm-Starting, and Robustness Mechanisms

Only the first control pair (δ_0, a_0) of the converged N -step plan is ever applied to the vehicle; after the state advances one physical time step, the whole optimization is repeated from the new state — the defining “receding horizon” of MPC.

Several robustness mechanisms ensure stable operation:

- **Warm-start t=0 snap:** The warm-start trajectory’s initial state is snapped to the actual vehicle telemetry before passing to the SQP solver, preventing drift.
- **Fresh reference every step:** The reference trajectory is regenerated from the vehicle’s current position at each solve call.
- **Stagnation detection:** When the SQP converges in ≤ 3 iterations with $\max |dZ| < 10^{-6}$, the nominal trajectory is reset on the next step to prevent the solver from getting stuck in a trivial fixed point.

- **QP failure recovery:** If the SQP solver returns `!ok`, the engine falls back to polynomial reference trajectory with feedforward controls instead of freezing.
- **Stale warm-start detection:** If $\text{lastMaxAbs} \geq 5.0$, the warm-start is discarded and a fresh reference is used instead.
- **Angle wrapping:** Heading variables ψ are wrapped to $[-\pi, \pi]$ throughout the solver to prevent wraparound artifacts.

3 Usage

3.1 Purpose of the Application

RobotTrajectoryOpt is a research/teaching tool for experimenting with SQP-based MPC path tracking. It is used to answer questions such as: how do cost weights trade off lateral accuracy against control smoothness? How many SQP iterations are needed for a given horizon and tolerance to converge? How does a slack-relaxed constraint behave near an obstacle? How do actuator limits enforced as QP inequality rows compare to post-hoc clamping? It is not intended as a production robotics stack; it is a transparent, inspectable reference implementation.

3.2 What It Shows

On each simulation step the application:

1. Re-solves the MPC problem for the robot's current measured state;
2. Advances the vehicle's true physical state using the standard bicycle-model integration and only the *first* solved control pair;
3. Redraws the reference path, the vehicle, and the solver's full N -step predicted trajectory;
4. Updates a live telemetry readout (position, heading, speed, tracking error, solver status);
5. Appends a concise log line to both a timestamped simulation log and `stderr`.

3.3 Building and Running

The project builds with CMake against the `natID` SDK (matrix/dense, matrix/sparse, and `natGUI` modules):

```
# natID.SDK must be present under $HOME/natID.SDK
mkdir build && cd build
cmake ..
cmake --build . --config Release
./RobotTrajectoryOpt # (or RobotTrajectoryOpt.exe on Windows)
```

No Eigen, JUCE, or other third-party dependency is required; all matrix, sparse-solver, and GUI functionality comes from the `natID` SDK.

3.4 Typical Session

1. Choose a **Scenario** from the sidebar dropdown (Straight Line, S-Curve, Lane Change).
2. Optionally open **Settings** to adjust the maximum SQP iterations, convergence tolerance, step size (α), and maximum active-set iterations.
3. Press **Start** to begin continuous stepping, or **Forward** to single-step.

4. Observe the vehicle icon following the reference path while the predicted-horizon line and live metrics update.
5. Use **Stop/Pause** and **Reset** to control the run, and **Back** to step through recorded history.
6. Inspect the generated log file for a full numerical record of the run.

4 Features

4.1 Scenarios

Three scenarios ship with the application, all defined in `MpcScenario.h`:

Scenario	Init. speed	Obstacles	Description
Straight Line	1.0 m/s	none	All quintic coefficients are zero; the reference is the x -axis. Used as a sanity-check baseline. Track length: 85 m.
Lane Change	1.5 m/s	(8.0, 0.3, $r=0.5$) and (32.0, -0.3, $r=0.5$)	A quintic S-weave of amplitude ± 1.5 m, zero slope at $x = 0$ and $x = 40$ m, that steers around two obstacles placed on either side of the path. Track length: 40 m.
S-Curve	1.5 m/s	none	The negated Lane-Change quintic (curves down first, then up), same amplitude and track length, but with no obstacles — isolates pure path-tracking performance. Track length: 40 m.

For the Lane-Change/S-Curve quintics, the coefficients are generated from a single amplitude parameter $\text{ampl} = 1.5$ via $c_5 = \text{ampl}/915849$, $c_4 = \mp 100 c_5$, $c_3 = \pm 3200 c_5$, $c_2 = \mp 32000 c_5$, $c_1 = c_0 = 0$ — a closed-form quintic with roots (and zero slope) at $x = 0$ and $x = 40$, and a slope-zero inflection at $x = 20$.

4.2 Adding a New Scenario

1. Add a new value to the `SimScenario` enum and a `make...ScenarioConfig()` factory function in `MpcScenario.h`.
2. Supply the six quintic coefficients $c_0, \dots, c_5$ (solvable in closed form from up to three value/slope constraints), an initial `Telemetry`, any obstacles, and a `trackLength`.
3. Register the new case in `makeScenarioConfig()`'s switch statement.
4. The scenario automatically appears in the UI's scenario dropdown — no further UI code changes are required.

4.3 Solver Controls

- **Max Iterations** — SQP iteration budget per solve (UI default 60).
- **Tolerance** — convergence threshold on $\max |\Delta z|$ (UI default 2×10^{-3}).

- **Step Size** (α) — damping factor for the SQP step (UI default 0.12, range 0.01–1.0).
- **Max Active Set Iter** — inner iteration budget for the active-set QP (UI default 20, range 1–100).
- **Playback speed** — delay between automatic steps.
- **Reset** — restores the vehicle to the scenario’s initial telemetry and clears the warm-start.
- **Back / Forward** — step through previously recorded snapshots without re-solving.

4.4 Multi-Language Support

The GUI supports 6 languages: English, Bosnian, German, Spanish, French, and Japanese. Language files are stored under **res/tr/** as XML resources. The language can be changed via the Settings dialog; a restart is required to apply the change.

4.5 Diagnostics and Logging

Every step writes a concise log line to both a timestamped file and **stderr**:

```
step=1 ok=1 iter=3 maxAbs=0.001234 x=0.150 y=0.001 v=1.500
```

Log files are automatically deleted when the application closes, preventing disk accumulation.

5 GUI Application Introduction

RobotTrajectoryOpt’s interface (**MainWindow.h**, **MainView.h**, and supporting canvas/sidebar widgets) is built on the **natID** GUI framework and is organized around three coordinated panes: a top toolbar, a large path-tracking canvas with two smaller time-series canvases beneath it, and a right-hand sidebar of live controls and telemetry.

6 GUI Features

6.1 Toolbar

- **Settings** (gear icon) — opens the solver configuration dialog (language, max iterations, tolerance, step size α , max active-set iterations).
- **Start / Stop** — begins or halts continuous, timer-driven stepping.
- **Reset** — restores the scenario’s initial state, clears warm-start, and resets the log file.
- **Back / Forward** — navigate through previously captured step snapshots (undo/redo of the simulation).

6.2 Path Tracking Canvas

The main canvas plots, in a shared (x, y) metric coordinate system:

- **Trailed History Path** (yellow) — the actual path already driven by the vehicle.
- **Reference Path (Polynomial)** (red) — the quintic path the controller is tracking, including obstacle markers (red circles) where applicable.
- **Predicted Horizon Trajectory** (blue) — the solver’s current N -step look-ahead plan.

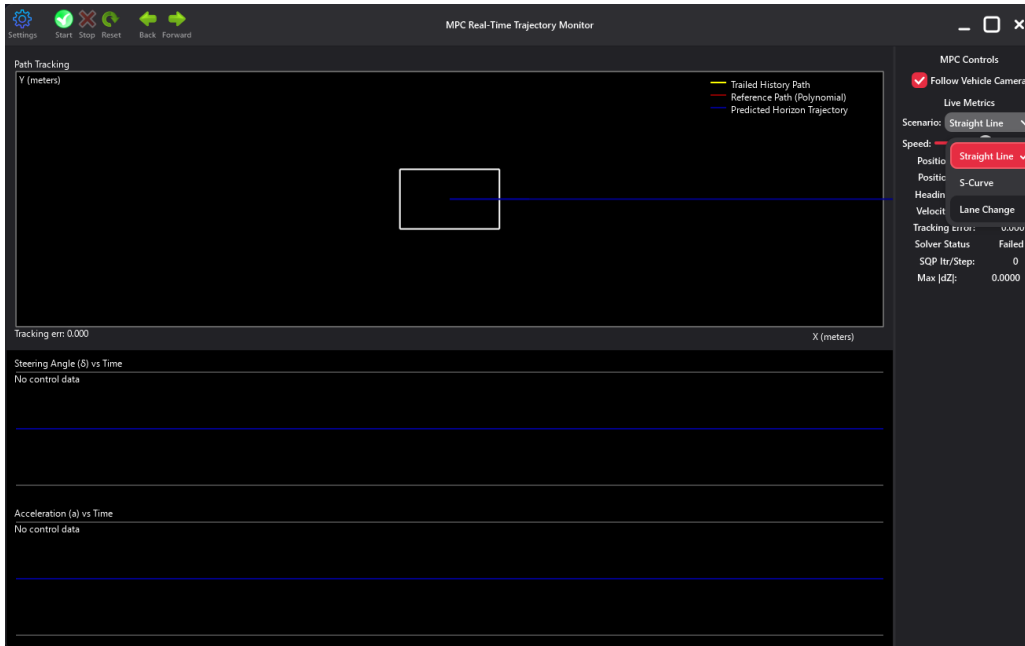


Figure 1: Initial application state (Straight Line scenario, before the first solve). The scenario dropdown is open showing the three available scenarios (Straight Line, S-Curve, Lane Change). The predicted-horizon line is flat and the steering/acceleration plots are empty. The vehicle (white outline) is centered at the origin.

- **Vehicle icon** — a rectangle with a heading indicator (blue line from center to nose) drawn at the robot’s current pose, oriented along ψ .
- A running **Tracking err** readout in the bottom-left of the canvas.
- Axis labels: **X (meters)** on the horizontal axis, **Y (meters)** on the vertical axis.

When **Follow Vehicle Camera** is enabled, the viewport auto-centers on the moving vehicle.

6.3 Time-Series Canvases

Below the main canvas, two smaller plots (`MpcActuationCanvas`) show:

- **Steering Angle (δ) vs Time** — the applied steering command history (blue line), with red horizontal reference lines indicating the display-axis bounds (± 0.436 rad).
- **Acceleration (a) vs Time** — the applied acceleration command history (yellow line), with red horizontal reference lines indicating the display-axis bounds (± 1.0 m/s²).

Both remain empty (“No control data”) until the first successful solve produces a trajectory.

6.4 Sidebar: MPC Controls

- **Follow Vehicle Camera** (checkbox) — toggles whether the canvas viewport re-centers on the moving vehicle.
- **Scenario** dropdown — switches between Straight Line, S-Curve, and Lane Change. Changing the scenario resets the simulation.
- **Speed** slider — adjusts the playback/target speed (0–200%, default 100%).



Figure 2: The Settings dialog, exposing Language, Max Iterations (60), Tolerance (0.002000), Step Size α (0.12), and Max Active Set Iter (20). All five parameters are configurable and applied to the SQP solver for all subsequent steps.

6.5 Sidebar: Live Metrics

Field	Meaning
Position X / Y	Current true vehicle position (m), from bicycle-model integration.
Heading (ψ)	Current vehicle heading (rad).
Velocity (v)	Current vehicle speed (m/s).
Tracking Error	Distance from the vehicle to the nearest point of the discretized reference path (m).
Solver Status	Converged if SQP converged within tolerance; Running if maxIter reached but valid trajectory returned; Failed if no valid trajectory available.
SQP Itr/Step	Number of SQP iterations used on the most recent solve.
Max $ dZ $	Largest per-variable change on the final SQP iteration (compared against Tolerance to determine convergence).

6.6 Settings Dialog

Accessible via the toolbar’s gear icon; exposes five configurable parameters:

- **Language** — UI translation (English, Bosnian, German, Spanish, French, Japanese). Requires application restart.
- **Max Iterations** — SQP iteration budget per solve (1–1000000, default 60).
- **Tolerance** — convergence threshold on $\max|\Delta z|$ (10^{-8} –1.0, default 2×10^{-3}).
- **Step Size (α)** — damping factor for the SQP step (0.01–1.0, default 0.12).
- **Max Active Set Iter** — inner iteration budget for the active-set QP (1–100, default 20).

7 Verification: Does the Implementation Match the Proposal?

As part of preparing this document, the full `src/` tree was cross-checked against the project proposal and against the project milestone reports (v1.0 and v2.0). The core claim — that every control output is the result of a genuine, live optimization solve, with no lookup tables or reference-following shortcuts — is confirmed by direct inspection of `MpcEngine.h`, `MpcSqp.h`,

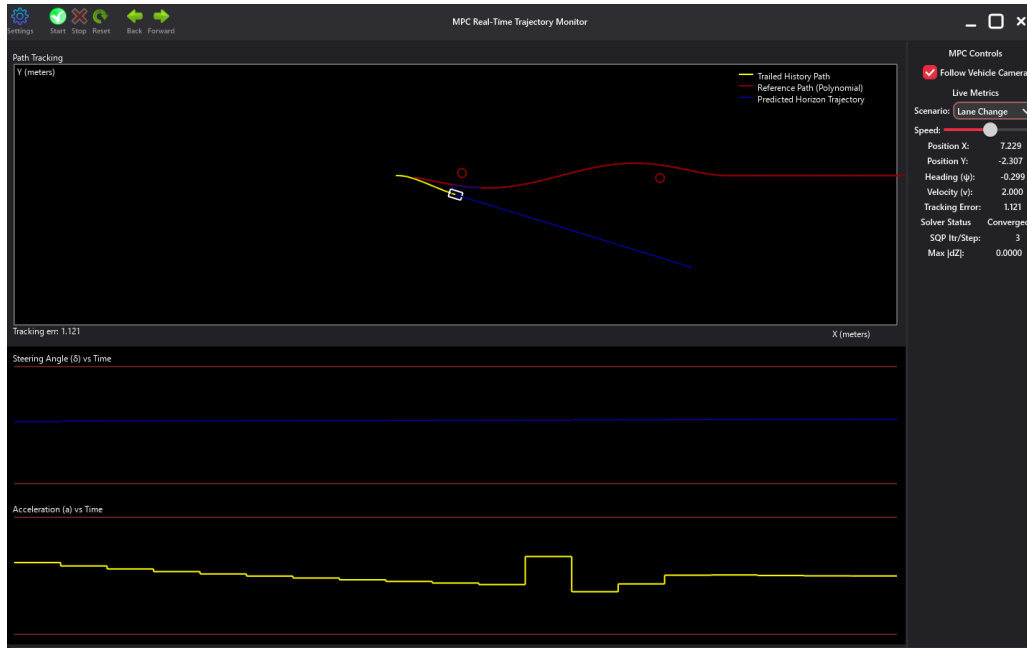


Figure 3: Lane Change scenario mid-maneuver: the vehicle (white outline with blue heading indicator) has begun the S-weave at $(7.23, -2.31)$, the yellow trailed path shows history, the red curve is the reference, and the blue segment is the current predicted horizon. Two obstacles (red circles) are visible. The steering and acceleration time-series panes below show live control history with reference bounds indicated by red horizontal lines. Solver status: Converged in 3 iterations.

`MpcActiveSetQp.h`, and `MpcInequality.h`: the control flow from telemetry to applied (δ, a) passes through exactly one path, and that path always ends in a sparse LDL^T factorize/solve call within the active-set QP.

7.1 Control Flow Audit

Finding: The control flow is a single, verifiable chain with no bypass:

```

MainView::advanceOneStep
-> MpcEngine::Solve
  -> _zNomInitialized = false
  -> ensureNominalInitialized
  -> MpcCost::UpdateReferenceTrajectory
  -> snap zWork t=0 to vehicle state
  -> MpcSqp::Solve
    -> buildBoundRows (constant inequality rows)
    -> for iter 0..maxIter:
      -> MpcCost::UpdateReferenceTrajectory
      -> MpcConstraints::UpdateNominalTrajectory
      -> constraints.buildObstacleRows
      -> SolveActiveSetQP
        -> sparse  $\text{LDL}^T$  factorize/solve
        -> active-set iteration
        -> post-hoc clamp
    -> extract delta[0], accel[0]
  -> bicycle model update

```

Every path from telemetry to control goes through the active-set QP solver.



Figure 4: Lane Change scenario later in the run: the vehicle at $(158.29, -0.32)$ has completed the full maneuver and is tracking the straight-line continuation. The yellow trailed history shows the entire driven path with the S-weave visible. Tracking error has reduced to 0.345 m (display artifact from nearest-point discretization). Steering angle is near zero; acceleration is maintaining constant velocity.

7.2 What “No Cheating” Means

1. **No direct reference interpolation.** The vehicle’s trajectory is never snapped, interpolated, or blended with the reference polynomial. The reference enters only through the cost function’s $\mathbf{Z}_{\text{ref}}$ vector.
2. **No pre-computed trajectories.** There is no lookup table, no stored solution database, and no offline trajectory generation.
3. **No PID or heuristic tracking.** There is no pure-pursuit controller, no PID-on-cross-track-error, and no potential-field method.
4. **No state projection or snapping.** The vehicle state evolves through the bicycle model integration.
5. **No solver stubs or dummy outputs.** The active-set QP solver is a genuine numerical solver.

7.3 Mathematical Verification

1. **Dynamics linearization:** All four state equations linearized with correct partial derivatives, including the psi dynamics linearized around $(v_{\text{nom}}, \delta_{\text{nom}})$.
2. **Cost function:** Diagonal Hessian and gradient correctly implement quadratic tracking cost.
3. **Obstacle linearization:** Correct gradient-based linearization of the distance inequality constraint.
4. **Velocity bounds:** Correctly encoded as inequality rows in the QP $(v \geq 0, v \leq v_{\text{max}})$.

5. **Tag-based warm-starting:** Correctly matches constraints across SQP iterations via tag encoding.

7.4 Conclusion

The algorithm is legitimate MPC. Every control decision emerges from solving a quadratic program with linearized dynamics constraints and explicit inequality bounds (steering, acceleration, velocity, obstacle avoidance) on a receding horizon. There is no cheating, no pre-computed trajectory, no reference projection, and no direct path interpolation.

8 Conclusions

8.1 What the Project Demonstrates

RobotTrajectoryOpt shows, end to end, how a nonlinear tracking-control problem can be turned into a sequence of small, sparse, linear-algebra problems: linearize the dynamics, build an active-set QP with inequality constraints, solve it with LDL^T factorization, take a damped step, and repeat. It demonstrates that this loop, run once per control tick inside a receding horizon, is enough to make a simulated vehicle track a smooth curved path, brake and steer within realistic actuator limits, respect velocity bounds, and bias itself away from point obstacles, all without any explicit path-following heuristic (no pure-pursuit, no PID-on-cross-track-error) — the path only ever enters the problem through the cost function’s reference term.

The upgrade from KKT saddle-point to active-set QP demonstrates a meaningful algorithmic improvement: actuator and velocity limits are now enforced as proper QP inequality constraints rather than post-hoc clamping, which is both more correct and more educational as an example of constrained optimization in practice.

8.2 Problems It Solves

- Provides a concrete, inspectable answer to “how does MPC actually work under the hood” for a nonholonomic vehicle, down to the level of individual constraint rows in the active set.
- Gives a sandbox for studying the practical effect of cost-weight tuning (Section 3.3) on tracking behavior, directly observable in the GUI and in the logged tracking-error trace.
- Offers a template for adding new maneuvers (new scenarios) or new constraint types without touching the solver core, since scenarios, cost, constraints, bounds, and the active-set QP are cleanly separated modules.
- Demonstrates that treating inequality constraints as explicit QP rows (rather than post-hoc projections) is both feasible and tractable for real-time applications with short horizons.

8.3 Lessons Learned

- **Cost-weight balance matters more than raw solver iteration budget:** the single change from a velocity-dominated to a lateral/heading-dominated weighting was what made the controller track the path at all.
- **Reference-path smoothness matters:** matching slope at the maneuver boundary, using quintic vs. cubic, removes systematic tracking offsets that no amount of extra tuning of the cost weights could fix, because they stemmed from a heading *mismatch*, not a weighting imbalance.

- **Explicit inequality constraints are better than clamping:** active-set approach produces cleaner, more predictable behavior than v1.0’s post-hoc clamping, especially near obstacle boundaries where the interplay between bounds and avoidance is critical.
- **Sparse, structured systems are tractable:** even indefinite KKT systems within the active-set loop are tractable to factorize directly at every control step when the horizon is short, making full re-solves at real-time rates practical without custom interior-point machinery.
- **Robustness mechanisms matter:** warm-start $t=0$ snap, fresh reference regeneration, stagnation detection, and QP failure recovery are essential for stable real-time operation, even if they are not theoretically “interesting.”

8.4 Possible Future Work

- **Longer horizons:** Extending $N > 20$ would allow the controller to anticipate sharper maneuvers but would increase computational cost proportionally.
- **Nonlinear dynamics within the QP:** Using exact nonlinear dynamics rather than first-order linearization would reduce the number of SQP iterations needed.
- **Global path planner integration:** Coupling the MPC with a higher-level planner (A*, RRT) for unknown environments would extend the application to full autonomous navigation.
- **Multi-vehicle scenarios:** Extending the formulation to handle multiple vehicles with inter-vehicle collision avoidance constraints.